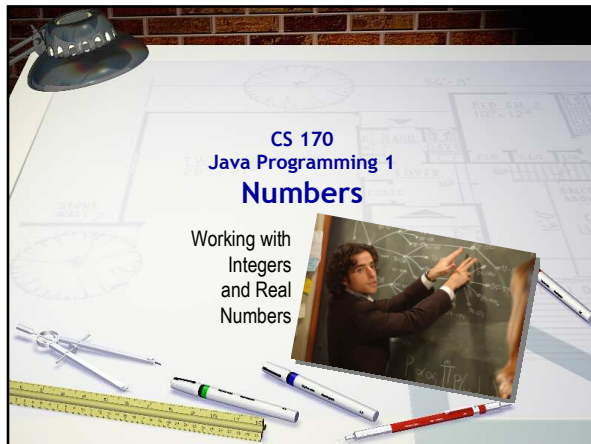
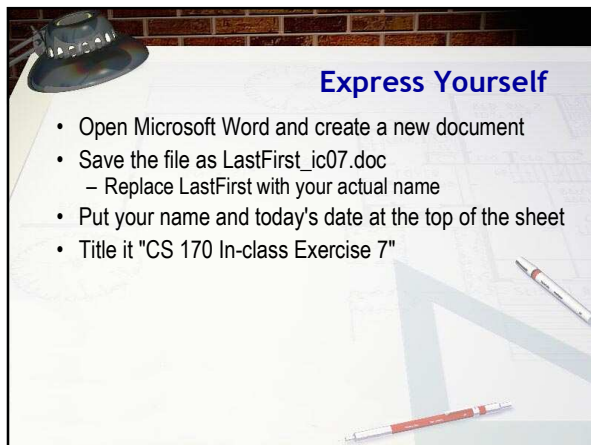
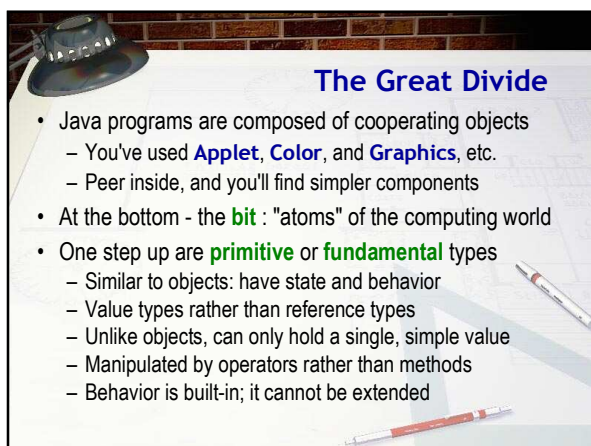


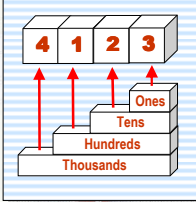






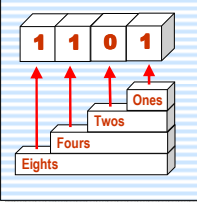
Decimal Numbers

- Internally, computers use **binary** to represent everything from whole numbers to real numbers to text to graphics
 - A binary number uses base 2 instead of base 10
- In **base 10**, or **decimal**:
 - We use ten digits, the digits 0 to 9
 - Each digit's value is based on its **positional value**
 - 4123 is 3 ones, 2 tens, 1 hundreds and four thousands



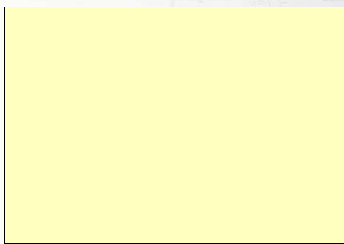
Binary Numbers

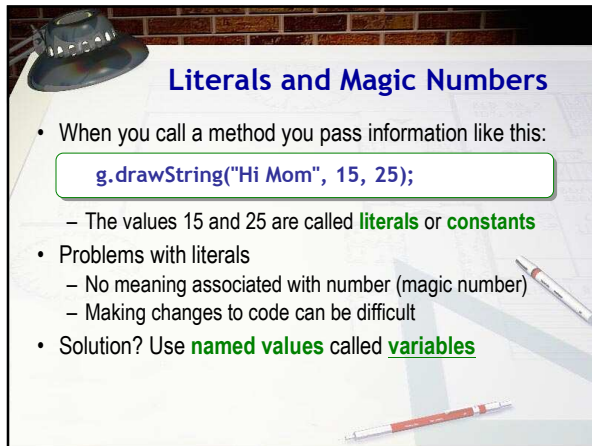
- In the base 2 or **binary** number system
 - We use only two digits 0 and 1
 - Instead of the tens, hundreds, and thousands places, which are powers of ten, each place value is a power of two
 - 1101 represents 1 one, 0 twos, 1 four and 1 eight (or 13)



Explain Yourself

- Exercise 1:** In binary, how do you represent these?
 - (Assume that these are decimal numbers)
 - 3
 - 5
 - 10
 - 8
 - 130
 - 255



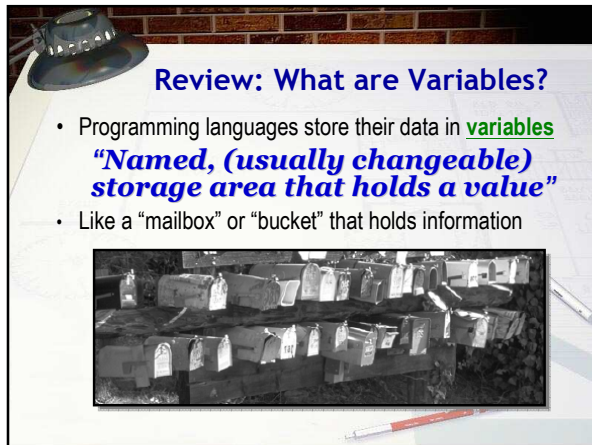


Literals and Magic Numbers

- When you call a method you pass information like this:

```
g.drawString("Hi Mom", 15, 25);
```

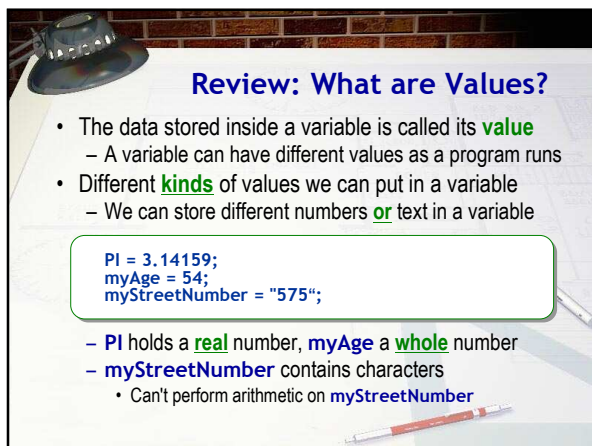
 - The values 15 and 25 are called **literals** or **constants**
- Problems with literals
 - No meaning associated with number (magic number)
 - Making changes to code can be difficult
- Solution? Use **named values** called **variables**



Review: What are Variables?

- Programming languages store their data in **variables**
"Named, (usually changeable) storage area that holds a value"
- Like a "mailbox" or "bucket" that holds information





Review: What are Values?

- The data stored inside a variable is called its **value**
 - A variable can have different values as a program runs
- Different **kinds** of values we can put in a variable
 - We can store different numbers **or** text in a variable

```
PI = 3.14159;  
myAge = 54;  
myStreetNumber = "575";
```

- PI** holds a **real** number, **myAge** a **whole** number
- myStreetNumber** contains characters
 - Can't perform arithmetic on **myStreetNumber**

Review: What are data types?

- Different **kinds** of variables for different **kinds** of data
- Different **sizes** of containers for **same kind** of data



Review: Creating Variables

- You can define (or create) variables in two places:
 - Inside a method : called a **local variable**
 - Outside a method : called a **field** or **instance variable**
- You define a local variable using this syntax:

type name = initial-value;

 - Parts in gray are optional.
 - The parts in red are required



Constants

- Named constants are “variables that don’t vary”
 - Make code clearer by eliminating **magic numbers**
 - Create by adding modifier **final** in front of declaration
 - For instance variables, use **public static final**
 - Convention is to use **UPPER_CASE** for names

```
8 public class TempConverter
9 {
10     //-----
11     // Computes the Fahrenheit equivalent of a specific Celsius
12     // value using the formula F = (9/5)C + 32.
13     //-----
14     public static void main (String[] args)
15     {
16         final int BASE = 32;
17         final double CONVERSION_FACTOR = 9.0 / 5.0;
18
19         double fahrenheitTemp;
20         int celsiusTemp = 24; // value to convert
21
22         fahrenheitTemp = celsiusTemp * CONVERSION_FACTOR + BASE;
23     }
24 }
```



Review: Assignment

- Values are placed in variables using **assignment**

```
int num = 23; // initializing assignment  
num = 45;    // executable assignment
```

- Value can be from **input**, a **literal**, or an **expression**

```
num = readInt("Enter a whole number");  
PI = 3.14159;  
area = PI * radius * radius;
```

Interactive Programs

- So far, our programs have been **static**, *not* dynamic
 - They work **exactly** the same each time they run
- We make our programs dynamic or interactive by having them process input supplied by the user
 - Input can be commands (mouse, menu or keyboard)
 - Input can also be data, which we'll store in variables
- The basic pattern for interactive programs:
 - Create a variable to hold the user's input
 - Ask the user to enter some data (called a **prompt**)
 - Read** the data, convert it, and **store** it in the variable

The Scanner Class

- The **Scanner** class simplifies console input
 - Import the **java.util.Scanner** class
 - Create a **Scanner** variable using **System.in**
 - Call **nextLine()**, and store input in a **String** variable

```
1 import java.util.Scanner;  
2  
3 public class Echo {  
4     // Reads a character string from the user and prints it.  
5     //-----  
6     public static void main (String[] args) {  
7         Scanner scan = new Scanner (System.in);  
8  
9         System.out.println ("Enter a line of text:");  
10  
11         String message = scan.nextLine();  
12  
13         System.out.println ("You entered: \"" + message + "\"");  
14     }  
15 }  
16
```

Express Yourself

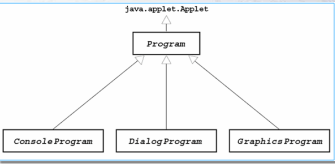
- **Exercise 2:** Create an interactive program that asks the user for their name, and then replies, "Hello <name>, I think I love you!". Use this pattern:
 - Create a BlueJ Console program
 - Make sure you import the `java.util.Scanner` class
 - Create a `Scanner` variable in your program
 - Create a `String` variable to hold the user's name
 - Ask the user to enter their name
 - Read the name using the `Scanner` object
 - Print the reply, using the name entered by the user

The ACM Class Libraries

- The ACM Java libraries were designed for students
 - ACM: Association for Computing Machinery
 - Computer Science Professional and Academic Organization
 - Student membership is about \$20 per year
 - <http://www.acm.org>
- Java Task Force (JTF) – Eric Roberts (Stanford) chair
 - Review the Java language
 - Develop a stable collection of libraries for teaching
 - <http://jtf.acm.org>
- We'll use several classes from the library

The ACM Program Classes

- **Problem 1:** Java
 - Can stick a `main`
 - Can't access in
 - Not very object
- **Problem 2:** Applet
- **Solution:** ACM program classes
 - Simple and OO like applets
 - Can be run as both applets or applications
 - Change input style by changing a single line of code
 - Both console-mode and graphics versions



```

classDiagram
    class Applet["java.applet.Applet"]
    class Program
    class ConsoleProgram["ConsoleProgram"]
    class DialogProgram["DialogProgram"]
    class GraphicsProgram["GraphicsProgram"]
    Applet <|-- Program
    Program <|-- ConsoleProgram
    Program <|-- DialogProgram
    Program <|-- GraphicsProgram
  
```

ACM Input

- ACM programs have their own input methods
 - Part of the **ConsoleProgram** class
- To read a line of text from the keyboard use **readLine()**

```
7
8
9
10 import acm.program.*;
11
12 public class ACMEcho extends ConsoleProgram
13 {
14     // Reads a character string from the user and prints it.
15     // Reads a character string from the user and prints it.
16     public void run ()
17     {
18         String message = readLine("Enter a line of text: ");
19         println ("You entered: \"" + message + "\"");
20     }
21 }
```



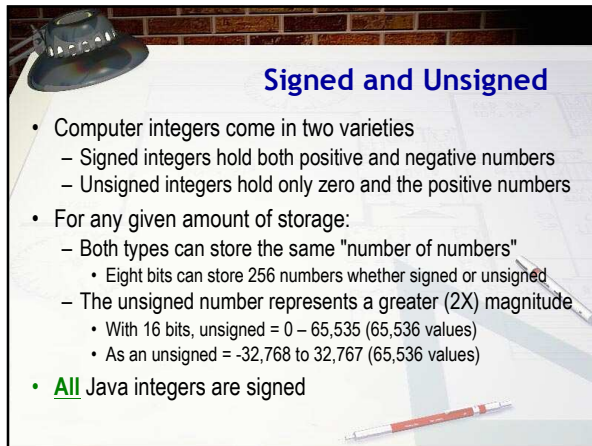
Express Yourself

- **Exercise 3:** Create the same program, using the ACM ConsoleProgram class. Use this pattern:
 - Create a BlueJ ACM Console program
 - Create a **String** variable to hold the user's name
 - Read the user's name using the **readLine()** method
 - Don't need a separate prompt line. Just ask the user to enter their name when calling **readLine()**
 - Print the reply, using the name entered by the user

Meet

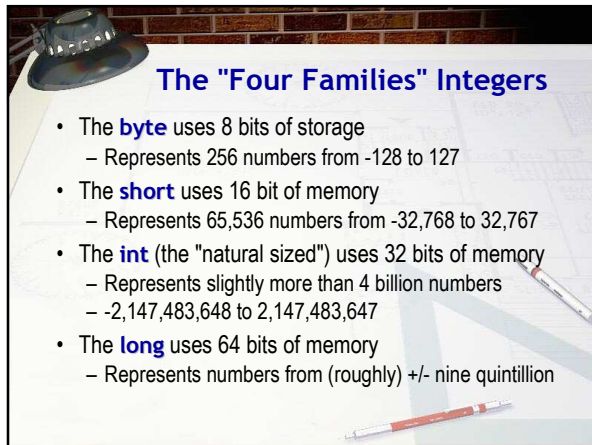
- The first primitive types we'll look at
- **Whole numbers**, including zero and negative numbers
 - 27 and -345 are integers, 7.25 is not
- Theoretically, integers are infinite
 - No matter how big, there's always room for another digit
- Computer-implemented integer primitives are **finite**
 - Stored in fixed area of memory, so different "sizes"
 - Represents "integer concept", limited by constraints
 - Similar to the way B/W photo represents reality





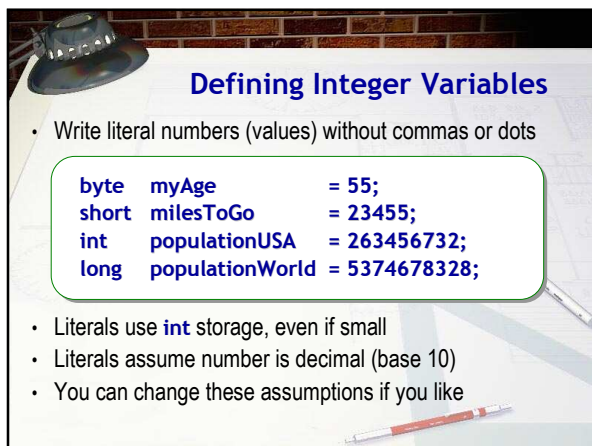
Signed and Unsigned

- Computer integers come in two varieties
 - Signed integers hold both positive and negative numbers
 - Unsigned integers hold only zero and the positive numbers
- For any given amount of storage:
 - Both types can store the same "number of numbers"
 - Eight bits can store 256 numbers whether signed or unsigned
 - The unsigned number represents a greater (2X) magnitude
 - With 16 bits, unsigned = 0 – 65,535 (65,536 values)
 - As an unsigned = -32,768 to 32,767 (65,536 values)
- **All** Java integers are signed



The "Four Families" Integers

- The **byte** uses 8 bits of storage
 - Represents 256 numbers from -128 to 127
- The **short** uses 16 bit of memory
 - Represents 65,536 numbers from -32,768 to 32,767
- The **int** (the "natural sized") uses 32 bits of memory
 - Represents slightly more than 4 billion numbers
 - -2,147,483,648 to 2,147,483,647
- The **long** uses 64 bits of memory
 - Represents numbers from (roughly) +/- nine quintillion

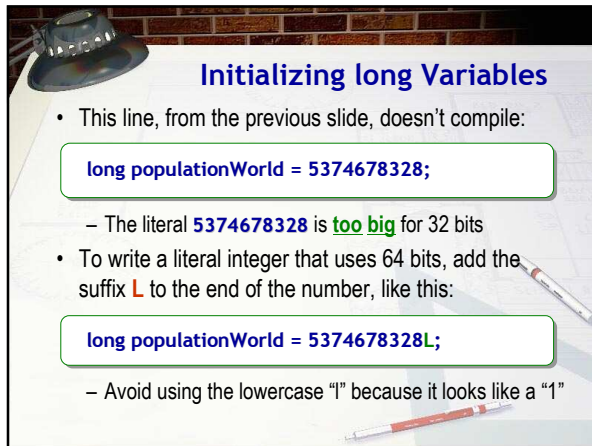


Defining Integer Variables

- Write literal numbers (values) without commas or dots

```
byte  myAge      = 55;  
short milesToGo  = 23455;  
int   populationUSA = 263456732;  
long  populationWorld = 5374678328;
```

- Literals use **int** storage, even if small
- Literals assume number is decimal (base 10)
- You can change these assumptions if you like



Initializing long Variables

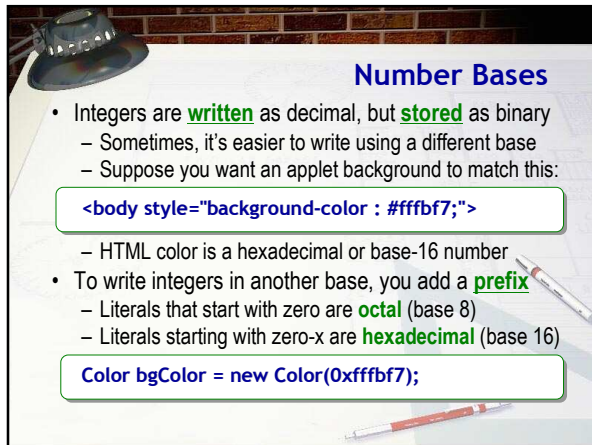
- This line, from the previous slide, doesn't compile:

```
long populationWorld = 5374678328;
```

 - The literal **5374678328** is **too big** for 32 bits
- To write a literal integer that uses 64 bits, add the suffix **L** to the end of the number, like this:

```
long populationWorld = 5374678328L;
```

 - Avoid using the lowercase "l" because it looks like a "1"



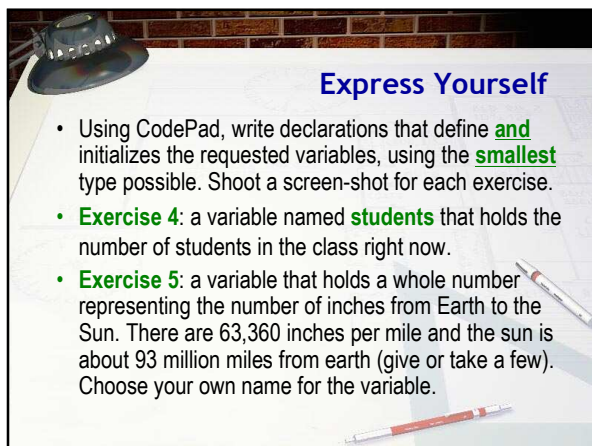
Number Bases

- Integers are **written** as decimal, but **stored** as binary
 - Sometimes, it's easier to write using a different base
 - Suppose you want an applet background to match this:

```
<body style="background-color : #ffbf7;">
```

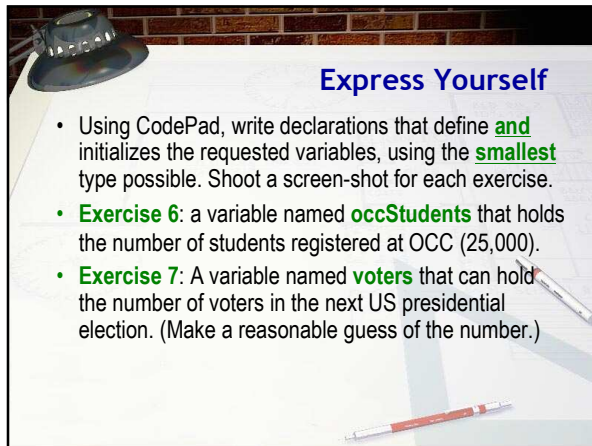
 - HTML color is a hexadecimal or base-16 number
- To write integers in another base, you add a **prefix**
 - Literals that start with zero are **octal** (base 8)
 - Literals starting with zero-x are **hexadecimal** (base 16)

```
Color bgColor = new Color(0xffbf7);
```



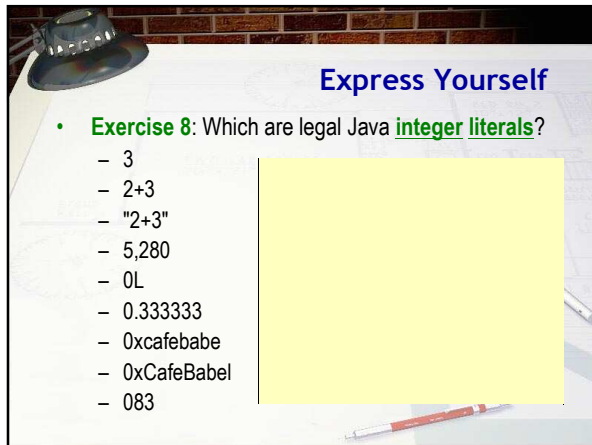
Express Yourself

- Using CodePad, write declarations that define **and** initialize the requested variables, using the **smallest** type possible. Shoot a screen-shot for each exercise.
- Exercise 4:** a variable named **students** that holds the number of students in the class right now.
- Exercise 5:** a variable that holds a whole number representing the number of inches from Earth to the Sun. There are 63,360 inches per mile and the sun is about 93 million miles from earth (give or take a few). Choose your own name for the variable.



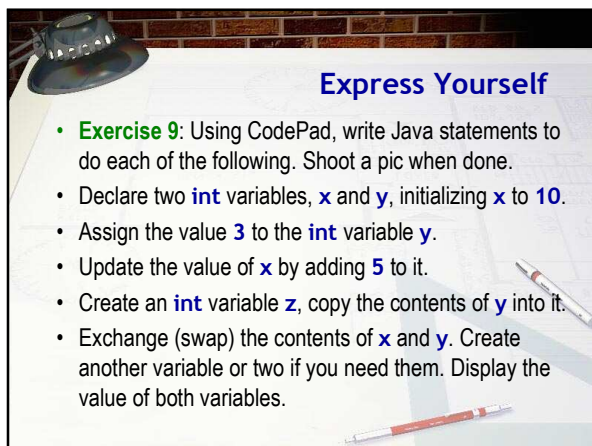
Express Yourself

- Using CodePad, write declarations that define **and** initialize the requested variables, using the **smallest** type possible. Shoot a screen-shot for each exercise.
- Exercise 6:** a variable named **occStudents** that holds the number of students registered at OCC (25,000).
- Exercise 7:** A variable named **voters** that can hold the number of voters in the next US presidential election. (Make a reasonable guess of the number.)



Express Yourself

- Exercise 8:** Which are legal Java **integer literals**?
 - 3
 - 2+3
 - "2+3"
 - 5,280
 - 0L
 - 0.333333
 - 0xcafebabe
 - 0xCafeBabel
 - 083



Express Yourself

- Exercise 9:** Using CodePad, write Java statements to do each of the following. Shoot a pic when done.
- Declare two **int** variables, **x** and **y**, initializing **x** to **10**.
- Assign the value **3** to the **int** variable **y**.
- Update the value of **x** by adding **5** to it.
- Create an **int** variable **z**, copy the contents of **y** into it.
- Exchange (swap) the contents of **x** and **y**. Create another variable or two if you need them. Display the value of both variables.

Integer Wrap-around

- **Exercise 10:** What values would you expect? Use CodePad to find out and shoot a screen-shot.

$(400000 * 400000 / 400000) == ????$
 $2147483647 + 1 == ????$
- Integers **wrap-around** or **overflow** when too big
- Which integer type should you use in your programs?
 - Unless you have a reason not to, use the **int**
 - Smaller sizes won't necessarily save memory
 - Calculations always use **int** or **long**

Integer Input

- The integer value 123 in your program is stored like this:

– 0000-0000 0000-0000 0000-0000 0111-1011
- When user enters 123 at the keyboard, it is **not** a number
 - Instead it is three characters '1', '2', and '3', like this:

– 0000-0000 0011-0001
 0000-0000 0011-0010
 0000-0000 0011-0011
- Integer input must **convert** from one to the other

Scanner Input

- You can use the **Scanner** class to read **and** convert
 - Create an **int** variable (**miles**) and a **Scanner** object
 - "Prompt" the user for the correct kind of input
 - Use the **Scanner's** **nextInt()** method to read

```

14 // ...
15 public static void main (String[] args)
16 {
17     int miles;
18     double gallons, mpg;
19
20
21
22
23
24

```

C:\WINDOWS\System32\cmd.exe

C:\Documents17895\user\83>java GeoMileage

Enter the number of miles: 5.25

Exception in thread "main" java.util.InputMismatchException

at java.util.Scanner.throwFor(Unknown Source)

at java.util.Scanner.next(Unknown Source)

at java.util.Scanner.nextInt(Unknown Source)

at java.util.Scanner.nextInt(Unknown Source)

at GeoMileage.main(GeoMileage.java:23)

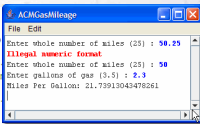
Express Yourself

- **Exercise 11:** Open BlueJ. You can create a new project if you like, or use a previous project. Create a class named EX07A using the Console Program template. Ask the user to enter two integers, read the result from the console, add them together and display the sum appropriately labeled. Run and snap two screen-shots, one of your code and one of the output.

ACM Console Input

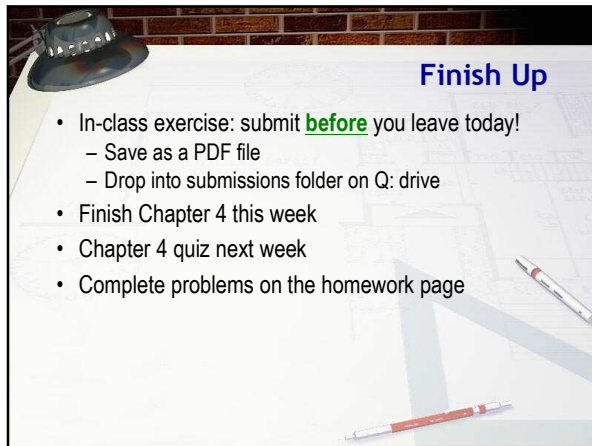
- ACM programs use the method named `readInt()`
 - Provide a prompt string when calling the method
 - “Catches” runtime errors and asks the user to re-enter

```
11 import acm.program.*;
12
13 public class ACMGasMileage extends ConsoleProgram {
14     //-----
15     // Calculates fuel efficiency based on
16     // user input.
17     //-----
18     public void run() {
19         int miles = readInt("Enter whole number of miles (25) : ");
20         double gallons = readDouble("Enter gallons of gas (3.5) : ");
21         double mpg = miles / gallons;
22         println("Miles Per Gallon: " + mpg);
23     }
24 }
```



Express Yourself

- **Exercise 12:** Create a class named EX07B using the ACM Console template. Ask the user to enter two integers, read the result from the console, add them together and display the sum. Run and snap two screen-shots, one of your code and one of the output.



Finish Up

- In-class exercise: submit **before** you leave today!
 - Save as a PDF file
 - Drop into submissions folder on Q: drive
- Finish Chapter 4 this week
- Chapter 4 quiz next week
- Complete problems on the homework page
